

kotlin

- [kotlin](#)
 - [probar código en línea](#)
 - [Instalación manual \(opcional\)](#)
 - [instalar IntelliJ Idea](#)
 - [definición de paquetes e importaciones](#)
 - [función main](#)
 - [Salida estándar](#)
 - [entrada estándar](#)
 - [funciones](#)
 - [variables](#)
 - [clases e instancias](#)
 - [herencia](#)
 - [comentarios](#)
 - [string templates](#)
 - [condicionales](#)
 - [if como expresión](#)
 - [ciclos](#)
 - [while](#)
 - [when](#)
 - [rangos](#)
 - [chequear si un número está fuera de rango](#)
 - [iterar sobre un rango](#)
 - [iterar sobre colecciones](#)
 - [chequear si una colección contiene un objeto usando in](#)
 - [usar expresiones lambda para filtrar y mapear colecciones](#)
 - [valores nulos](#)
 - [programación funcional](#)
 - [función de orden superior](#)
 - [funciones lambda](#)
 - [map](#)
 - [filter](#)
 - [reduce](#)
 - [fold \(reduce con valor inicial\)](#)
 - [crear aplicación](#)
 - [ligas](#)

Kotlin es un lenguaje de tipado estático que corre sobre la JVM (Java Virtual Machine). Su primera versión estable salió en 2016, y desde entonces ha ganado una popularidad enorme, especialmente después de que Google lo declarara lenguaje oficial para Android en 2017. sintaxis básica

probar código en línea

[probar código en línea](#)

Instalación manual (opcional)

1. [instalar jvm \(opcional\)](#)
2. Instalar el compilador si desea trabajar desde un editor

3. [Compilador](#)
4. Busca el archivo kotlin-compiler-2.3.21.zip (o versión más reciente) y descarga
5. Descomprime el archivo en una carpeta, por ejemplo C:\Kotlin
6. Agrega kotlin al path
7. Verifica instalacion: `kotlinc -version`

instalar IntelliJIdea

[instalar](#)

Al crear un nuevo proyecto: file/new/proyect, elija kotlin en lugar de java

definicion de paquetes e importaciones

La definición de paquete debe ser la primera línea

```
package my.demo

import kotlin.text.*

// ...
```

funcion main

```
fun main() {
    println("Hello world!")
}
```

```
fun main(args: Array<String>) {
    println(args.contentToString())
}
```

Salida estandar

```
print("Hello ")
print("world!")
```

entrada estandar

```
// Prints a message to request input
println("Enter any word: ")

// Reads and stores the user input. For example: Happiness
val yourWord = readln()

// Prints a message with the input
```

```
print("You entered the word: ")
print(yourWord)
// You entered the word: Happiness
```

funciones

```
fun sum(a: Int, b: Int): Int {
    return a + b
}
```

El cuerpo de una función puede ser una expresión, en ese caso se infiere el tipo de dato que se devuelve:

```
fun sum(a: Int, b: Int) = a + b
```

Una función que no devuelve ningún valor

```
fun printSum(a: Int, b: Int): Unit {
    println("sum of $a and $b is ${a + b}")
}
```

se puede omitir el tipo de retorno Unit

```
fun printSum(a: Int, b: Int) {
    println("sum of $a and $b is ${a + b}")
}
```

variables

```
// declara una variable de solo lectura con el valor 5
val x: Int = 5
// 5
// Declara una variable x y la inicializa con 5
var x: Int = 5
// se puede reasignar un nuevo valor a la variable x
x += 1
// 6
```

Kotlin soporta inferencia de tipos

```
// Declara la variable x con valor de 5;el tipo `Int`es inferido
val x = 5
// 5
```

Puede utilizar variables sólo después de inicializarlas. Puede inicializar una variable en el momento de la declaración o declarar una variable primero e inicializarla más tarde. En el segundo caso, deberás especificar el tipo de datos.

```
// Inicializa x en el momento de la declaración; el tipo no es requerido
val x = 5
// Declara la variable c sin inicialización; el tipo si es requerido
val c: Int
// Inicializa la variable despuesd e la declaración
c = 3
```

Se puede declarar variables en el nivel supeerior.

```
val PI = 3.14
var x = 0

fun incrementX() {
    x += 1
}
// x = 0; PI = 3.14
// incrementX()
// x = 1; PI = 3.14
```

clases e instancias

Las propiedades de una clase se pueden enumerar en su declaración o cuerpo:

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
```

El constructor predeterminado con los parámetros enumerados en la declaración de clase está disponible automáticamente.

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
fun main() {
    val rectangle = Rectangle(5.0, 2.0)
    println("El perimetro es ${rectangle.perimeter}")
}
```

herencia

La herencia entre clases se declara mediante dos puntos (:). Las clases son definitivas de forma predeterminada; para que una clase sea heredable, márkuela como abierta.

```
open class Shape

class Rectangle(val height: Double, val length: Double): Shape() {
```

```
    val perimeter = (height + length) * 2
}
```

comentarios

```
// This is an end-of-line comment

/* This is a block comment
   on multiple lines. */
```

string templates

```
var a = 1
// simple name in template:
val s1 = "a is $a"

a = 2
// arbitrary expression in template:
val s2 = "${s1.replace("is", "was")}, but now is $a"
```

condicionales

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) {
        return a
    } else {
        return b
    }
}
```

if como expresión

```
fun maxOf(a: Int, b: Int) = if (a > b) a else b
```

ciclos

```
val items = listOf("apple", "banana", "kiwifruit")
for (item in items) {
    println(item)
}
```

otra forma:

```
val items = listOf("apple", "banana", "kiwifruit")
for (index in items.indices) {
    println("item at $index is ${items[index]}")
}
```

while

```
val items = listOf("apple", "banana", "kiwifruit")
var index = 0
while (index < items.size) {
    println("item at $index is ${items[index]}")
    index++
}
```

when

```
fun describe(obj: Any): String =
    when (obj) {
        1          -> "One"
        "Hello"    -> "Greeting"
        is Long    -> "Long"
        !is String -> "Not a string"
        else       -> "Unknown"
    }
```

rangos

```
val x = 10
val y = 9
if (x in 1..y+1) {
    println("fits in range")
}
```

chechar si un número esta fuera de rabgo

```
val list = listOf("a", "b", "c")

if (-1 !in 0..list.lastIndex) {
    println("-1 is out of range")
}
if (list.size !in list.indices) {
    println("list size is out of valid list indices range, too")
}
```

iterar sobre un rango

```
for (x in 1..5) {  
    print(x)  
}  
for (x in 1..10 step 2) {  
    print(x)  
}  
println()  
for (x in 9 downTo 0 step 3) {  
    print(x)  
}
```

iterar sobre colecciones

```
for (item in items) {  
    println(item)  
}
```

chechar si una colección contiene un objeto usando in

```
when {  
    "orange" in items -> println("juicy")  
    "apple" in items -> println("apple is fine too")  
}
```

usar expresiones lambda para filtrar y mapear colecciones

```
val fruits = listOf("banana", "avocado", "apple", "kiwifruit")  
fruits  
    .filter { it.startsWith("a") }  
    .sortedBy { it }  
    .map { it.uppercase() }  
    .forEach { println(it) }
```

valores nulos

Una referencia debe marcarse explícitamente como anulable cuando sea posible un valor nulo. ¿Tienen nombres de tipos anulables? al final. Por ejemplo, Int?.

Devuelve nulo si str no contiene un número entero:

```
fun parseInt(str: String): Int? {  
    return str.toIntOrNull()
```

```
}
```

programación funcional

función de orden superior

```
// Función que recibe otra función como parámetro
fun operacionMatematica(a: Int, b: Int, operacion: (Int, Int) -> Int): Int {
    return operacion(a, b)
}

// Uso con diferentes operaciones
val suma = operacionMatematica(5, 3) { x, y -> x + y }
val multiplicacion = operacionMatematica(5, 3) { x, y -> x * y }
val maximo = operacionMatematica(5, 3) { x, y -> maxOf(x, y) }

println(suma) // 8
println(multiplicacion) // 15
println(maximo) // 5
```

funciones lambda

```
val numeros = listOf(1, 2, 3, 4, 5)

// Lambda tradicional
val cuadrados = numeros.map { numero -> numero * numero }

// Con "it" (parametro implícito cuando hay solo uno)
val cuadradosSimplificado = numeros.map { it * it }

// Múltiples lambdas en una función
numeros.forEach { numero ->
    println("Número: $numero")
}

println(cuadrados) // [1, 4, 9, 16, 25]
```

map

```
val precios = listOf(100.0, 200.0, 300.0)

// Aplicar descuento del 10%
val conDescuento = precios.map { it * 0.9 }
println(conDescuento) // [90.0, 180.0, 270.0]

// Convertir a string formateado
```

```
val facturas = precios.map { "${it}" }
println(facturas) // [$100.0, $200.0, $300.0]
```

filter

```
val edades = listOf(12, 15, 18, 22, 30, 17)

val mayoresDeEdad = edades.filter { it >= 18 }
println(mayoresDeEdad) // [18, 22, 30]

// Combinación map + filter
val nombres = listOf("Ana", "Luis", "Alexander", "Eva")
val nombresLargos = nombres.filter { it.length > 3 }.map { it.uppercase() }
println(nombresLargos) // [ANA, LUIS, ALEXANDER]
```

reduce

```
val ventas = listOf(100, 200, 50, 300)

// Suma total
val total = ventas.reduce { acumulado, actual -> acumulado + actual }
println(total) // 650

// Producto total
val producto = ventas.reduce { acumulado, actual -> acumulado * actual }
println(producto) // 100 * 200 * 50 * 300 = 300,000,000
```

fold (reduce con valor inicial)

```
val numeros = listOf(1, 2, 3)

// Suma empezando desde 10
val sumaInicial = numeros.fold(10) { acumulado, actual -> acumulado + actual }
println(sumaInicial) // 16 (10 + 1 + 2 + 3)

// Construir string
val resultado = numeros.fold("Números: ") { acumulado, actual ->
    "$acumulado $actual"
}
println(resultado) // "Números:  1 2 3"
```

crear aplicación

En IntelliJ.

1. new/proyect/ elija lenguaje kotlin/ ponga el nombre y seleccione carpeta
2. en la carpeta src, abra el archivo Main.kt
3. ejecute con ►

ligas

[referencia](#)